

ELTE EÖTVÖS LORÁND UNIVERSITY

INTRODUCTION TO BLAZOR

Modern web application development in .NET

Lecture 8

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

WHAT IS BLAZOR?

Blazor is a framework for building multiplatform interactive web applications based on .NET Core and C#.

- Enables developers to write client-side web UI using C# instead of JavaScript or TypeScript.
- Supports various hosting models for rendering
  - the *Blazor WebAssembly* uses *WebAssembly*
  - the *Blazor Server* uses *SignalR*
- Provides a *component-based architecture*, making it easier to build and manage UI elements efficiently
- Part of *ASP.NET Core*, benefiting from the security, performance, and scalability of the .NET ecosystem

WHAT IS WEBASSEMBLY?

WebAssembly (*WASM*) is a binary instruction format designed for running high-performance applications in web browsers and other environments.

- WebAssembly is designed as a portable, efficient, and secure execution environment that runs across different platforms.
- Supported by all modern browsers, including *Chrome*, *Firefox*, *Edge*, and *Safari*.
- *Blazor WebAssembly* (*Blazor WASM*) allows C# code to run in the browser by compiling it into WebAssembly.
- Uses .NET runtime compiled to WebAssembly, enabling full .NET applications in the browser without requiring JavaScript.

WHY USE BLAZOR?

- Allows *full-stack* development using C#
  - Desktop and hybrid application development is also supported (e.g. MAUI, WPF, Avalonia)
- Eliminates the need for JavaScript in many scenarios
  - Can interoperate with JavaScript, allowing integration with existing JavaScript libraries if needed
- Provides rich UI components and two-way data binding
  - Supports real-time updates and two-way data binding, making the UI more interactive with less effort
- Integrates well with existing .NET libraries

BLAZOR HOSTING MODELS

Blazor applications can run in 3 major ways:

1. **Blazor WebAssembly (WASM)**
   - Runs in the browser (or webview) using *WebAssembly*
   - Executes .NET code client-side
2. **Blazor Server**
   - Runs on the server, sending UI updates via *SignalR*
   - Requires constant connection to the server
3. **Blazor Hybrid (via .NET MAUI or WPF)**
   - Runs inside native desktop and mobile applications
   - Enables building cross-platform apps with Blazor

BLAZOR APPLICATION ARCHITECTURE

Blazor app typically follows a three-layered architecture that includes:

1. **Presentation Layer (UI)**
   - contains Razor components that define UI elements
   - handles routing, event handling, and data binding
2. **Business Logic Layer (Services)**
   - contains application logic and business rules
   - provide services like: API calls, state management, validation
3. **Data Access Layer (DAL)**
   - Blazor Server: connects to a database via e.g. EF Core
   - Blazor WebAssembly: calls APIs for data retrieval

COMPONENT-BASED ARCHITECTURE

Blazor applications are built using *Razor Components*, which allow developers to create interactive web UIs using C# and .NET.

- Blazor provides a modern alternative to JavaScript-based frameworks by leveraging .NET technologies.
- Blazor applications are structured using *components*, which are reusable UI elements written in .razor files.
- Each component can include *HTML*, *C#*, and *Razor* syntax, allowing for dynamic content and event handling.
- Components can be nested inside other components, making UI development modular and maintainable.

COMPONENT-BASED ARCHITECTURE

- e.g. (*Blazor component*):

```
<h3>Hello, @Name!</h3>
<ul>
  @foreach (var item in Items)
  {
    <li>@item</li>
  }
</ul>

@code {
  string Name = "Blazor";
  string[] Items = { "...", "...", "..." }
}
```

EVENT HANDLING

Razor components have built-in event handling

- uses **`@on[event]`** syntax, e.g.:

```
<h3>Counter Example</h3>
<p>Current count: @count</p>
<button @onclick="IncrementCount">Increment</button>

@code {
  int count = 0;
  void IncrementCount() {
    count++;
  }
}
```

EVENT HANDLING

Razor components can handle various user interactions and browser events. Some common event types in Blazor:

- mouse events: **`@onclick`**, **`@onmouseenter`**, **`@onmouseover`**, etc.
- keyboard events: **`@onkeydown`**, **`@onkeyup`**, **`@onkeypress`**
- form events: **`@onchange`**, **`@onsubmit`**, etc.
- focus events: **`@onfocus`**, **`@onblur`**
- touch events: **`@ontouchstart`**, **`@ontouchend`**, **`@ontouchmove`**, etc.
- clipboard events: **`@oncop`**, **`@onpaste`**, **`@oncut`**
- etc.

RENDERING & UI UPDATES

- Blazor automatically updates the UI based on changes in data
- When the state of a component changes, Blazor re-renders only the affected parts of the page, optimizing performance
- Blazor does not re-render the entire page when a change occurs
    - Instead, it uses a Virtual DOM diffing algorithm (similar to *React*), to efficiently update the UI

RENDERING & UI UPDATES

- e.g. when the **count** variable is modified, Blazor detects the change and updates only the **<p>** tag instead of reloading the entire page:

```
<h3>Counter Example</h3>
<p>Current count: @count</p>
<button @onclick="IncrementCount">Increment</button>

@code {
  private int count = 0;
  void IncrementCount() {
    count++; // Blazor automatically updates the UI
  }
}
```

RENDERING & UI UPDATES

- when working with lists of data, use the **@key** directive to optimize rendering, preventing unnecessary renders:

```
<ul>
  @foreach (var item in Items)
  {
    <li @key="item.Id">@item.Name</li>
  }
</ul>

@code {
  Product[] Items = …
}
```

DATA BINDING

Blazor supports one-way and two-way data binding, allowing components to interact with user input

- *One-way binding* updates the UI based on a C# property
- *Two-way binding* keeps the UI and the underlying C# property in sync
- e.g.:

```
<input @bind="name" />
<p>Hello, @name!</p>

@code {
    string name = "Blazor";
}
```

COMPONENT PARAMETERS

Component parameters allow data to be passed from a parent component to a child component

- This enables reusable and dynamic components that can behave differently based on the provided data
- Parameters are defined using the **`[Parameter]`** attribute inside the child component
- The **`[EditorRequired]`** attribute can mark a parameter as required.
- e.g. (**`ParentComponent.razor`**):

```
<h3>Parent Component</h3>
<ChildComponent Title="Welcome to Blazor!" Number="5" />
```

COMPONENT PARAMETERS

- e.g. (**ChildComponent.razor**):

```
<h3>Child Component</h3>
<p>Title: @Title</p>
<p>Number: @Number</p>

@code {
    [Parameter]
    public string Title { get; set; }
    [Parameter]
    public int Number { get; set; }
}
```

COMPONENT PARAMETERS

- sometimes, child components need to send data back to the parent
  - this is achieved using an EventCallback, which allows a child component to notify its parent when an event occurs.
  - e.g. (**`ChildComponent.razor`**):

```
<button @onclick="SendMessage">Send Message to Parent</button>
@code {
  [Parameter]
  public EventCallback<string> OnMessageReceived { get; set; }

  async Task SendMessage() {
    await OnMessageReceived.InvokeAsync("Hello from Child!");
  }
}
```

COMPONENT PARAMETERS

- e.g. (**ParentComponent.razor**):

```
<h3>Parent Component</h3>
<p>Message from Child: @message</p>

<ChildComponent OnMessageReceived="HandleMessage" />

@code {
  private string message = "No message yet";
  void HandleMessage(string msg) {
    message = msg;
  }
}
```

ROUTING

Routing in Blazor allows the application to navigate between different components or views based on the URL.

- It is a key feature for building *single-page applications* (SPAs), where the page does not reload when navigating between different views.
- Blazor uses a *router* to map URLs to components, and each component can represent a different page or view in the app.

ROUTING

- The router is defined in the **App.razor** file and listens for changes in the browser's address bar to determine which component to render.

```
<App>
  <Router AppAssembly="@typeof(Program).Assembly">
    <Found Context="routeData">
      <RouteView RouteData="@routeData"
                 DefaultLayout="@typeof(MainLayout)" />
    </Found>
    <NotFound>
      <p>The page you're looking for could not be found.</p>
    </NotFound>
  </Router>
</App>
```

ROUTING

- Each Blazor component that should be a routable page must include the **@page** directive at the top.
- This directive binds the component to a specific URL, which is used by the router to determine which component to load.
  - e.g.:

```
@page "/home"
<h3>Hello, @Name!</h3>

@code {
  string Name = "Blazor";
}
```

ROUTING

- Blazor allows components to accept *dynamic route parameters*, similar to traditional URL query strings or route segments.
- These parameters can be accessed in the component using the **`[Parameter]`** attribute.
  - e.g.:

```
@page "/product/{Id}"
<h3>Product Details</h3>
<p>Product ID: @Id</p>

@code {
  [Parameter] public int Id { get; set; }
}
```

ROUTING

- Constraints to restrict the route parameters can also be define.
  - e.g.:

```
@page "/product/{Id:int:min(1)}"
<h3>Product Details</h3>
<p>Product ID: @Id</p>

@code {
  [Parameter] public int Id { get; set; }
}
```

- Blazor supports nested routing. Components can be nested within each other and routed based on the parent component's URL.

NAVIGATION

Navigation among pages can be achieved via the **`NavLink`**

- **`NavLink`** is similar to an anchor **`<a>`** tag but provides additional functionality, such as *active link styling*
- e.g.:

```
<NavLink href="/home" class="nav-link">Home</NavLink>
<NavLink href="/about" class="nav-link">About</NavLink>
```

- when the user clicks on these links, the app will navigate to the corresponding routes without causing a full page reload

NAVIGATION

- alternatively, **`NavigationManager`** allows for navigation from C# code
- e.g.:

```
@inject NavigationManager Navigation
@* dependency injection *@

<button @onclick="GoToHome">Go to Home</button>

@code {
  private void GoToHome()
  {
    Navigation.NavigateTo("/home");
  }
}
```